

Data Science and ML Placement Roadmap

A focused plan for Amandeep Singh based on current GitHub projects and the ML, NLP, Data Science, Data Engineering, and RAG/LLM job descriptions.

Main goal: Become placement-ready for fresher roles such as Data Scientist Intern, ML Engineer Intern, Applied AI Intern, Data Analyst with ML, and junior AI/ML developer.

Core strategy: Do not restart machine learning from zero. Use your existing ML foundation, then add the missing layer: NLP, LLM/RAG, deployment, SQL, and interview explanation.

1. What The Companies Are Really Asking

The screenshots show different types of roles. They use similar words like ML, AI, RAG, Python, APIs, and data, but they are not the same job.

Role Type	What They Expect	Your Fit	Action
Machine Learning Engineer Fresher	Python, preprocessing, feature engineering, regression, classification, clustering, evaluation, overfitting, API deployment.	Strong base because of your Bank Churn and ML projects.	Revise deeply and deploy one project.
Data Scientist / Applied AI Intern	ML, NLP, text processing, LLM APIs, Streamlit demos, SQL, model evaluation, business interpretation.	Best target for you.	Add NLP and RAG projects.
Data Engineer Intern	SQL, ETL, data pipelines, databases, data quality, cloud basics.	Possible, but this is less ML-focused.	Keep as backup. Strengthen SQL and ETL basics.
AI/ML Architect, RAG, Multi-Agent	RAG, vector databases, LangChain, agents, cloud, FastAPI, memory, orchestration.	Advanced for a fresher, but project exposure can help.	Build one practical RAG project. Do not try to master everything.

Your best target: Data Scientist Intern / ML Engineer Intern / Applied AI Intern.

Your positioning line: "I can clean and analyze data, build classical ML models, explain metrics, work with SQL, and create deployed AI/ML demos using NLP and RAG."

2. Your Skill Blocks

You should organize your preparation into six blocks. These are ordered by placement value.

Block A: Python for Data Science

You do not need advanced Python theory. You need practical coding confidence.

- Data structures: list, tuple, set, dict.
- Functions, lambda, map, filter, list comprehension.

- File handling: CSV, JSON, text files.
- Error handling: `try` , `except` , basic debugging.
- OOP basics: class, object, constructor, methods.
- Libraries: NumPy, pandas, matplotlib, seaborn, scikit-learn.

Placement depth: You should be able to solve basic Python questions and explain your project code without hesitation.

Block B: SQL and Databases

SQL gives very high return for both data science and data engineering roles.

- Basic queries: `SELECT` , `WHERE` , `ORDER BY` , `LIMIT` .
- Aggregations: `COUNT` , `SUM` , `AVG` , `GROUP BY` , `HAVING` .
- Joins: inner, left, right, full outer concept.
- Subqueries and correlated subqueries.
- CTEs using `WITH` .
- Window functions: `ROW_NUMBER` , `RANK` , `DENSE_RANK` , `LAG` , `LEAD` .
- Case statements and conditional aggregation.
- Basic database design: primary key, foreign key, normalization, star schema.

Project use: Add 20 to 30 business SQL questions to one dashboard project. Example: monthly revenue, top customers, churn by segment, top genres, repeat customers.

Block C: Classical Machine Learning

You already know ML, so revise for interview depth rather than watching a full course again.

- Supervised learning: regression and classification.
- Regression models: Linear Regression, Ridge, Lasso, ElasticNet.
- Classification models: Logistic Regression, KNN, Naive Bayes, SVM basics.
- Tree models: Decision Tree, Random Forest, Gradient Boosting, XGBoost.
- Unsupervised learning: KMeans, hierarchical clustering, DBSCAN basics, PCA basics.
- Feature engineering: scaling, encoding, missing value handling, outliers, datetime features.
- Model evaluation: accuracy, precision, recall, F1, ROC-AUC, PR-AUC, confusion matrix, MAE, MSE, RMSE, R2.
- Model selection: train-test split, cross-validation, hyperparameter tuning.
- Important concepts: bias-variance, overfitting, underfitting, regularization, data leakage, class imbalance.

Interview depth: For every algorithm, know when to use it, its weakness, and one real example from your project.

Block D: NLP Fundamentals

This is the biggest missing area in your profile based on the JDs.

- Text cleaning: lowercasing, punctuation removal, stopwords, stemming, lemmatization.
- Tokenization: sentence and word tokenization.
- Vectorization: Bag of Words, n-grams, TF-IDF.
- Text classification: sentiment analysis, spam detection, topic/category prediction.
- Named Entity Recognition basics: person, organization, location extraction.
- Topic modeling basics: LDA concept.
- Word embeddings: Word2Vec concept, sentence embeddings concept.
- Evaluation: accuracy, F1, confusion matrix for text classification.

Project target: Build one complete NLP project using TF-IDF plus Logistic Regression, Naive Bayes, or XGBoost. Deploy with Streamlit.

Block E: LLM, Embeddings, and RAG

This is what many modern AI JDs are asking. You do not need to become a research engineer. You need practical understanding and one strong demo project.

- LLM basics: what large language models do, tokens, context window, temperature.
- Prompt engineering: clear instructions, role prompts, examples, output formats.
- Embeddings: converting text into vectors for similarity search.
- Vector search: cosine similarity, nearest neighbor search.
- Vector databases: FAISS, Chroma, Qdrant, or pgvector basics.
- RAG pipeline: load documents, split text, create embeddings, store vectors, retrieve relevant chunks, generate answer.
- LangChain or LlamaIndex basics: document loaders, text splitters, retrievers, chains.
- RAG evaluation basics: relevance, hallucination, answer faithfulness, source citation.
- Agent basics: tool calling, ReAct idea, simple workflow automation. Keep this basic.

Project target: Build a Resume and Job Description Matcher or Chat With PDF app using RAG. This directly matches your placement JDs.

Block F: Deployment and Engineering

This separates job-ready candidates from notebook-only candidates.

- Streamlit: quick ML/NLP demo apps.
- FastAPI: create prediction API endpoints.
- Model saving: `joblib` , `pickle` .
- Project structure: `src/` , `app.py` , `requirements.txt` , `README.md` .
- Git and GitHub: clean commits, proper repo descriptions, pinned projects.
- Basic cloud idea: what AWS/GCP/Azure are, EC2, S3, Cloud Run basics. Do not go deep now.
- Optional Docker basics: what a container is, `Dockerfile` idea.

3. What To Build

You do not need many more projects. You need the right projects.

Project 1: Upgrade Bank Customer Churn

Purpose: Prove classical ML strength.

- Keep your existing model comparison and XGBoost result.
- Add a Streamlit app: user enters customer details and receives churn probability.
- Add SHAP or feature importance for interpretability.
- Add a simple FastAPI endpoint: `/predict`.
- Add a clean README with problem, dataset, workflow, metrics, screenshots, and how to run.

Interview story: "I handled class imbalance, compared multiple models, selected XGBoost using ROC-AUC and PR-AUC, and deployed it as a usable churn prediction app."

Project 2: NLP Text Classification

Purpose: Cover NLP fundamentals from JDs.

- Choose one dataset: movie reviews, product reviews, tweets, spam messages, or support tickets.
- Clean text and create TF-IDF features.
- Train Logistic Regression, Naive Bayes, Random Forest, and XGBoost if useful.
- Evaluate with confusion matrix, precision, recall, and F1.
- Deploy with Streamlit.

Suggested title: Customer Review Sentiment Analyzer.

Interview story: "I converted raw text into numerical features using TF-IDF, trained classification models, compared F1 scores, and deployed a sentiment prediction demo."

Project 3: RAG-Based Resume and JD Matcher

Purpose: Match modern AI, RAG, embeddings, LangChain, vector DB, and LLM keywords.

- User uploads resume and job description.
- App extracts text from PDF or text input.
- Create embeddings for resume sections and JD requirements.
- Use vector similarity to match skills and missing areas.
- Use an LLM to generate a structured report: match score, strong areas, missing skills, project suggestions, interview focus.
- Add source-based reasoning or show matched text snippets.

Suggested stack: Python, Streamlit, LangChain or LlamaIndex, FAISS or Chroma, OpenAI or Hugging Face embeddings, optional FastAPI.

Interview story: "I built a RAG app that retrieves relevant resume and JD sections using embeddings and generates a grounded skill-gap report."

4. 90-Day Placement Plan

Days	Focus	Output
1-10	ML revision, metrics, leakage, imbalance, interview questions from your Bank Churn project.	ML interview notes plus improved Bank Churn README.
11-20	SQL practice: joins, CTEs, window functions, business queries.	SQL practice repo or SQL section added to dashboard project.
21-35	NLP fundamentals and text classification project.	Deployed NLP Streamlit app.
36-55	LLM, embeddings, vector DB, RAG, LangChain basics.	RAG Resume and JD Matcher MVP.
56-65	Deployment: Streamlit, FastAPI, requirements, project structure.	Bank Churn app/API and RAG app cleaned.
66-75	Resume, GitHub profile, pinned repos, LinkedIn, project descriptions.	Placement-ready public profile.
76-90	Mock interviews, SQL, DSA, ML questions, project explanation practice.	Interview readiness and application routine.

5. Daily Routine

Use this routine during active preparation. Keep it realistic and repeatable.

- **45 minutes:** SQL practice.
- **45 minutes:** DSA or Python coding.
- **90 to 120 minutes:** project building.
- **30 minutes:** ML/NLP/LLM theory revision.
- **15 minutes:** write notes or update GitHub README.

If college work is heavy, reduce project time but do not skip SQL and coding completely.

6. Interview Topics You Must Prepare

ML Questions

- What is overfitting and how do you reduce it?
- What is data leakage? Give an example.
- Precision vs recall vs F1.
- Why is PR-AUC useful for imbalanced data?
- Why did you choose XGBoost in churn prediction?
- How does Random Forest reduce overfitting?
- Difference between bagging and boosting.
- How do you handle missing values?
- Why should preprocessing be inside a pipeline?
- How do you handle class imbalance?

NLP Questions

- What is tokenization?
- What is TF-IDF?
- Bag of Words vs TF-IDF.
- Stemming vs lemmatization.
- How do you build a sentiment classifier?
- Why can text data be sparse?
- What are embeddings?

LLM and RAG Questions

- What is an embedding?
- What is vector search?
- What is RAG?
- Why use RAG instead of only prompting an LLM?
- What is chunking?
- How do you reduce hallucination?
- What is LangChain used for?
- What is the difference between fine-tuning and RAG?

SQL Questions

- Second highest salary.
- Top 3 products by revenue per category.
- Customers who purchased in one month but not the next.
- Monthly revenue growth.
- Rank employees/customers/products using window functions.
- Difference between WHERE and HAVING.
- Inner join vs left join.

7. GitHub and Resume Positioning

Your GitHub already has good raw material. Now make it easier for recruiters to understand.

- Pin only your strongest 4 to 6 projects.
- Keep certificates lower priority than projects.
- Every flagship repo should have a clear description.
- Every flagship README should include problem statement, tech stack, workflow, results, screenshots, and how to run.
- Add live demo links wherever possible.
- Use metrics in project bullets, not generic lines.

Resume Project Bullets

- Built a bank churn prediction pipeline using XGBoost, handling class imbalance and achieving ROC-AUC 0.88 and PR-AUC 0.71.
- Developed an NLP sentiment classification app using TF-IDF and ML models, deployed with Streamlit for real-time predictions.
- Built a RAG-based resume and JD matcher using embeddings and vector search to identify skill gaps and generate structured recommendations.

- Designed SQL-driven dashboards using MySQL and Power BI to analyze business KPIs and trends.

8. What Not To Study Right Now

These topics appear in some JDs, but they are not the best use of time for fresher placement unless a specific company requires them.

- Deep learning from scratch in heavy detail.
- Advanced robotics and embedded systems.
- CUDA, Rust, C++ performance optimization.
- Advanced cloud architecture.
- Fine-tuning Llama/Mistral deeply.
- Too many new certificates.
- Another full beginner ML course.

9. Final Roadmap Summary

Phase 1: Revise classical ML deeply using your own Bank Churn project.

Phase 2: Strengthen SQL and basic DSA for placement filters.

Phase 3: Learn NLP basics and build one text classification app.

Phase 4: Learn LLM embeddings and RAG, then build one RAG project.

Phase 5: Deploy projects, polish GitHub, rewrite resume, and practice interviews.

Simple rule: If a topic does not improve your projects, resume, or interview performance in the next 90 days, keep it for later.